

CS 630: Software Testing and Quality Assurance

5-3 Paper: Recommending a Test Toolchain and Workflow

Malgorzata Debska

Southern New Hampshire University

Dr. Ben Torrales

August 5, 2026

Recommending a Test Toolchain and Workflow

LogiCo has recently transitioned from a monolithic application to a combination of a monolithic system and microservices. The company develops software for Web, Android, and iOS platforms using technologies such as Java Spring Boot, Node.js, Python, React, Oracle, Docker, and Jenkins. Although Jenkins automates part of the deployment process, the company still relies heavily on inconsistent manual testing because there is no dedicated QA team. As a result, LogiCo has experienced integration issues, production defects, and inconsistent test coverage. According to Heusser and Larsen (2023), a successful testing strategy combines automated and manual testing throughout the software development life cycle. Implementing a standardized testing toolchain will help LogiCo improve software quality while continuing to support frequent software releases.

Key Test Tooling Needs

LogiCo's biggest need is improving automated testing. Developers work in two-week sprints and merge code often, making manual testing alone too slow to keep up with continuous development. Automated testing provides immediate feedback and allows developers to identify problems before code is deployed. The company also needs stronger integration testing because several microservices communicate with each other and with third-party services such as Google Maps, FedEx, and Stripe. Every update should verify that these systems continue exchanging information correctly. Another important need is regression testing. When one part of the application changes, existing features such as billing, package tracking, route optimization, and user management should continue working without introducing new defects. Performance testing

is equally important because the application must remain stable during periods of heavy user activity. Heusser and Larsen (2023) explain that combining multiple testing methods provides better coverage and improves overall software quality.

Recommended Test Toolchain

To improve testing, I recommend using a combination of tools that work together throughout development. For feature validation and unit testing, developers should use JUnit for Java services and PyTest for Python services. These tests confirm that each new user story and individual function behaves as expected before code is merged into the project. For integration testing, I recommend Postman. Postman automatically verifies communication between LogiCo's microservices and external APIs. Since many business functions depend on data moving correctly between systems, integration testing helps identify communication problems before deployment. For regression testing, I recommend Selenium. Selenium automates common user activities such as logging in, creating shipments, tracking packages, and processing payments. Running these tests after every code change helps ensure that existing functionality continues working after new features are added. For performance testing, I recommend Apache JMeter. JMeter simulates hundreds of users accessing the application simultaneously and measures response times. This allows developers to identify performance issues before customers experience slow or unreliable service. Finally, Jenkins should continue serving as the company's continuous integration and deployment tool. Instead of running only unit tests, Jenkins should automatically execute unit, integration, regression, and performance tests whenever developers merge new code. As Heusser and Larsen (2023) explain, automated testing provides fast feedback and reduces the risk of releasing software with defects.

High-Level Testing Workflow

The testing workflow should begin during sprint planning. When the team receives a new feature or user story, developers should determine which automated tests will be needed before coding begins. Planning tests early help prevent gaps in test coverage later in development. During coding, developers should write unit tests alongside the application code. Before submitting a pull request, they should execute these tests locally to verify that the new feature works correctly. After the pull request is reviewed and approved, Jenkins should automatically start the testing pipeline. Unit tests should run first. If they pass, Postman should execute integration tests to verify communication between services and external APIs. Selenium should then perform regression testing to ensure existing features continue working correctly. Before major releases, JMeter should run performance tests to confirm the application performs well under expected workloads. If every automated test passes, the application should be deployed to the staging environment. Developers should then complete a final manual review by testing the most important business functions on the Web, Android, and iOS applications. Billing services should continue requiring management approval before deployment because they process regulated information. Once approval is received, the software can safely be released to production. After deployment, developers should continue monitoring application logs, system performance, and customer feedback. If a production issue is discovered, it should be fixed immediately, and a new regression test should be added to prevent the same issue from happening again. According to Heusser and Larsen (2023), continuously expanding automated tests improves software quality over time.

Justification for the Recommended Toolchain

The recommended toolchain fits LogiCo's technical environment because it builds tools the company already uses instead of replacing them. Jenkins is already part of the CI/CD pipeline, making it easy to expand automated testing without significantly changing the existing workflow. The workflow also fits the company's organizational structure. Since developers currently perform QA responsibilities, these tools allow testing to become part of their daily work instead of creating additional manual tasks. Automated testing reduces repetitive work, provides faster feedback, and identifies problems earlier in development. Heusser and Larsen (2023) explain that integrating testing throughout development improves software quality while supporting faster and more reliable software releases.

Supporting Future Growth

This testing strategy can easily grow as LogiCo continues expanding its software. As new microservices, APIs, or mobile features are introduced, developers can simply add new automated tests without changing the overall workflow. The same approach could also be used in healthcare, banking, insurance, retail, and manufacturing because these industries also depend on multiple systems communicating reliably. Heusser and Larsen (2023) note that standardized testing processes make it easier for organizations to scale while maintaining consistent software quality.

Conclusion

LogiCo's current manual testing process is no longer sufficient for a growing enterprise application. Implementing JUnit, PyTest, Postman, Selenium, JMeter, and Jenkins creates a

complete testing toolchain that supports feature validation, unit, integration, regression, and performance testing. Combining automated testing with manual reviews and approval steps will help the company reduce production defects, improve software quality, and support future growth. To conclude, Heusser and Larsen (2023) demonstrate that organizations using structured testing workflows are better prepared to deliver reliable software and maintain customer confidence.

References

Heusser, M., & Larsen, M. (2023). *Software testing strategies: A testing guide for the 2020s*.